Min-Heap Algorithms & Reference Manual

CLRS-COMPLIANT ALGORITHM SPECIFICATIONS, PROCEDURES & COMPLEXITY ANALYSIS

1. Helper & Indexing Functions

In standard binary heaps (1-based array indexing), explicit parent-child relations are computed in O(1) time using bitwise or integer operations:

```
PARENT(i) = ⌊i / 2⌋          LEFT(i) = 2i                RIGHT(i) = 2i + 1
```

2. Bottom-Up Min-Heap Construction — O(n) Time

Builds a min-heap in linear time by processing nodes backwards from index ⌊n / 2⌋ down to 1 using downward heapification (sift-down).

MIN-HEAPIFY(A, i) — O(log n)

```
1  l = LEFT(i)
2  r = RIGHT(i)
3  if l <= A.heap-size and A[l] < A[i]
4       smallest = l
5  else smallest = i
6  if r <= A.heap-size and A[r] < A[smallest]
7       smallest = r
8  if smallest != i
9       exchange A[i] with A[smallest]
10      MIN-HEAPIFY(A, smallest)
```

BUILD-MIN-HEAP(A) — O(n)

```
1  A.heap-size = A.length
2  for i = ⌊A.length / 2⌋ downto 1
3       MIN-HEAPIFY(A, i)
```

3. Top-Down Min-Heap Construction — O(n log n) Time

Constructs the min-heap incrementally by inserting elements one by one into an expanding heap boundary and floating items up (sift-up).

HEAP-DECREASE-KEY(A, i, key) — O(log n)

```
1  if key > A[i]
2       error "new key is larger than current key"
3  A[i] = key
4  while i > 1 and A[PARENT(i)] > A[i]
5       exchange A[i] with A[PARENT(i)]
6       i = PARENT(i)
```

MIN-HEAP-INSERT(A, key) — O(log n)

```
1  A.heap-size = A.heap-size + 1
2  A[A.heap-size] = ∞
3  HEAP-DECREASE-KEY(A, A.heap-size, key)
```

BUILD-MIN-HEAP-TOP-DOWN(A) (In-Place) — O(n log n)

```
1  A.heap-size = 1
2  for i = 2 to A.length
3       HEAP-DECREASE-KEY(A, i, A[i])
```

BUILD-MIN-HEAP-ONLINE(A) (Fresh Heap) — O(n log n)

```
1  n = A.length
2  A.heap-size = 0
3  for i = 1 to n
4       MIN-HEAP-INSERT(A, A[i])
```

4. Min-Priority Queue Supporting Procedures

HEAP-MINIMUM(A) — O(1)

```
1  return A[1]
```

HEAP-EXTRACT-MIN(A) — O(log n)

```
1  if A.heap-size < 1
2       error "heap underflow"
3  min = A[1]
4  A[1] = A[A.heap-size]
5  A.heap-size = A.heap-size - 1
6  MIN-HEAPIFY(A, 1)
7  return min
```

5. Comparative Analysis & Performance Summary

Construction Method	Primary Algorithm	Time Complexity	Auxiliary Space	Sifting Direction
Bottom-Up	`BUILD-MIN-HEAP`	**O(n)**	O(1)	Sifts down (`MIN-HEAPIFY`)
Top-Down	`BUILD-MIN-HEAP-TOP-DOWN`	**O(n log n)**	O(1)	Sifts up (`HEAP-DECREASE-KEY`)
Online Stream	`BUILD-MIN-HEAP-ONLINE`	**O(n log n)**	O(1) dynamic grow	Sifts up (`MIN-HEAP-INSERT`)